

# Machine Learning with System/Software Engineering in Selection and Integration of Intelligent Algorithms

Jasser Alharbi

Florida Institute of Technology  
Melbourne, Florida 32905  
jalharbi2019@my.fit.edu

Siddhartha Bhattacharyya

Florida Institute of Technology  
Melbourne, Florida 32905  
sbhattacharyya@fit.edu

**Abstract**—Machine learning has become an essential component in the design of intelligent systems across several disciplines. This widespread use of machine learning has led to the importance of evaluating how Systems/Software Engineering approaches go hand in hand with Machine Learning to reliably integrate intelligence in software systems. In this research effort, our motivation is to develop a systematic approach also termed as Machine Learning Engineering for the selection and integration of machine learning algorithms in systems. The proposed approach discusses combining a structured approach for designing and developing system/software with an experimental analysis that data scientists perform on machine learning algorithms. This experimental analysis is essential as some of the characteristics exhibited by intelligent algorithms cannot be predicted or guaranteed compared to systems without intelligent algorithms. In this paper, we elaborate on our system/software engineering guided disciplined approach by comparing two machine learning algorithms that focus on the recognition of handwritten digits. The algorithms we compare are the Logistic Regression and Neural Network algorithms. After the analysis, we identify the contracts that should be associated with intelligent components to better predict the behavior of the system as a result of the selection of one of the components to be a machine learning algorithm. Finally, we indicate how the results can be used by Systems/Software Engineers in integrating intelligent algorithms.

## I. INTRODUCTION

Machine learning has demonstrated tremendous improvement in performance over the last decade. This has created several opportunities for integrating machine learning algorithms within a system. As a result, it becomes very important to reliably integrate machine learning algorithms to have a trustworthy product. To achieve this objective, we discuss our structured approach which discusses the sequence of processes that need to be followed in a disciplined way in order to develop a reliable system. This structured approach is inherited from the principles of systems/software engineering. To elaborate upon our approach we evaluated several applications of machine learning algorithms such as, for vision, decision making and finally selected object recognition.

Object recognition is a technique that allows the computer to identify objects in images or videos and it is one of the most important applications investigated through deep learning and machine learning, as discussed by Ahmad et al [1]. It

is easy for people to understand and read the handwriting from documents, but challenging for machines. With the development of handwriting recognition technology, instead of making people rewrite documents into electronic formats the primary motivation is to automatically read from these documents. Manually rewriting, is expected to take long with more human effort. Additionally, manually rewriting might introduce more errors while copying. Object recognition has a part called Optical Character Recognition (OCR), which is the field that addresses the issues related to this problem of handwriting recognition. It is the area that studies many different aspects such as recognizing patterns, also image and vision processing, as a part of AI. OCR is a system that lets the machine read the text from image or handwriting document and converts all the characters to digital form. It is faster to retrieve data using OCR as well as it saves upon space [2].

Digit recognition can be used for several different kinds of application, such as reading checks in banks, recognizing street numbers, reading from handwritten notes etc [1]. Furthermore, with the increase in use of touchscreen devices and tablets, digit recognition is no longer limited to handwritten on paper, but is extended to recognition from screen to digital [3]. For an intelligent machine to recognize handwritten characters from diverse set of dataset, Machine Learning algorithms need to be applied for improving performance with precise recognition of characters. This elaborate understanding of the need for the intelligent application is the first step in system/software engineering towards understanding the capabilities that are enabled by the machine learning algorithms, which justifies the reason for integrating machine learning algorithms.

As these intelligent systems are integrated with autonomous systems or integrated into safety critical applications such as, for medical purposes or aerospace there needs to a more structured approach to assure quality and determine performance boundaries. To address this concern, a structured system/software engineering guided approach needs to be applied to release a stable and reliable intelligent system/software as a product. In our study, we explore and compare two different types of algorithms, that support handwriting recognition that are Logistic Regression and Neural Networks and discuss

the essential steps needed for the integration of machine learning algorithms in the software or system development lifecycle. Hulten [4] describes software engineering strategies for building intelligent systems by introducing the concept of Machine Learning engineering, we extend such an approach by identifying the performance boundaries that should be satisfied by the system or the intelligent component for stable performance.

In this paper, we provide background research effort that has been previously performed in section II. In section III we elaborate on a planned framework to compare machine learning algorithms, which is an essential integration of steps that need to be followed in order to deliver intelligent system/software as reliable product. The two algorithms we experimented with are further explained in section IV. Finally, in section V we discuss all the experiments performed, with the results discussed in section VI. This is followed by conclusion in section VII

## II. LITERATURE REVIEW

Comparing performance between machine learning algorithms is a way to decide which machine learning algorithm works the best for a use case. Deciding which algorithm to use is difficult because of the immense number of machine learning algorithms. Each algorithm will perform better than others in one field. Caruana and Niculescu-Mizil stated that “the different performance metrics measure different tradeoffs in the predictions made by a classifier, and it is possible for learning methods to perform well on one metric, but be suboptimal on other metrics” [5]. Caruana and Niculescu-Mizil compared ten algorithms for a variety of situations and evaluated each algorithm separately and found that each algorithm performed best with a problem but not with other problems [5]. Han et al compared machine learning algorithms in predicting the time for software project and they found different results from each machine learning algorithms [6].

Recognizing hand-written characters and digits are popular research in machine learning field. Also, text categorization is popular in processing natural language system in extracting data from documents [7]. Hand-written recognition performs better each year as the algorithms evolve. Some of the challenges that digit recognition machine learning algorithms face are in the size, shape and writing style of the characters and digits [8]. Moreover, Perwej and Chaturvedi found that Neural Network did a good job in recognizing hand-written characters with 82% accuracy [8]. Lewis et al examined two machine learning algorithms -Bayesian Classifier and a Decision Tree- in text categorization and found that both algorithms perform an acceptable performance [7].

Given the existence of research efforts by data scientist that provide insight into comparing machine learning algorithms, we need to integrate a structured approach to better understand the contract associated with the machine learning component of the intelligent system.

## III. PROPOSED STRUCTURED FRAMEWORK FOR MACHINE LEARNING ENGINEERING

The proposed Systems/Software Engineering Guided Machine Learning (SEGML) methodology should be a well planned interaction between a software engineer (SE) and a data scientist (DS). This is because, a SE focuses on delivering a reliable product and a DS focuses on performance. The first step of the methodology involves, selecting a use case scenario depending upon the application, then we need to evaluate the need for applying intelligence in the application. This is where, a SE needs to interact with a DS to develop an understanding of what type of intelligence is enabled by ML, which automation fails to enable. Following this step, we need to try to identify the machine learning algorithms that will support adding intelligence to the application. This is another step where DS can provide more technical information and practical experience with the potential algorithms. After that, we identify the dataset we will train and examine the machine learning algorithms on, which is again pointed to by data scientists. The next step is in identifying the key performance parameters (KPP) such as, learning rate, the layers in an algorithm etc. The identification of KPP is a result of informed discussion between the SE and DS, as a SE has the skill of abstracting relevant information to build models, whereas, a DS knows which parameters can be easily changed and which might require structural or in depth changes. Next, we evaluate the performance of each machine learning algorithm in accuracy and the time that the machine learning algorithms take to learn. Then, we change the parameters to see the difference in performance and we repeat the execution and the evaluation. The final outcome of the method are the assumptions and guarantees met by the machine learning algorithms, which are contracts satisfied by the machine learning component based on the values of KPPs. These contracts can be used for compositional analysis of the intelligent system. Additionally, when a trained intelligent algorithm is used these become the specifications thus, supporting the notions of reusability.

In applying the discussed methodology we selected the application area to be handwriting recognition, followed by the need to recognize the digits with machine learning. In the process, the requirements for the handwritten recognition was identified such as, to classify handwritten objects with more than 90% accuracy within  $10^{-5}$  seconds, with a batch size of 100 or less the classifier should classify with more than 90% accuracy, within a training time of  $10^{-5}$  seconds accuracy of classification should be more than 90%. The Input from DS helped identify that accurate classifications are possible even for low and high resolution inputs, which is not possible from automation without intelligence. In the next step, further discussion with DS supported the process of selecting the ML algorithms. The ML algorithms we chose were Logistic regression and Neural Networks, as these are popular algorithms used for handwritten recognition, which had the potential to meet the requirements, specified earlier. Once the ML algorithms were selected the next step was in

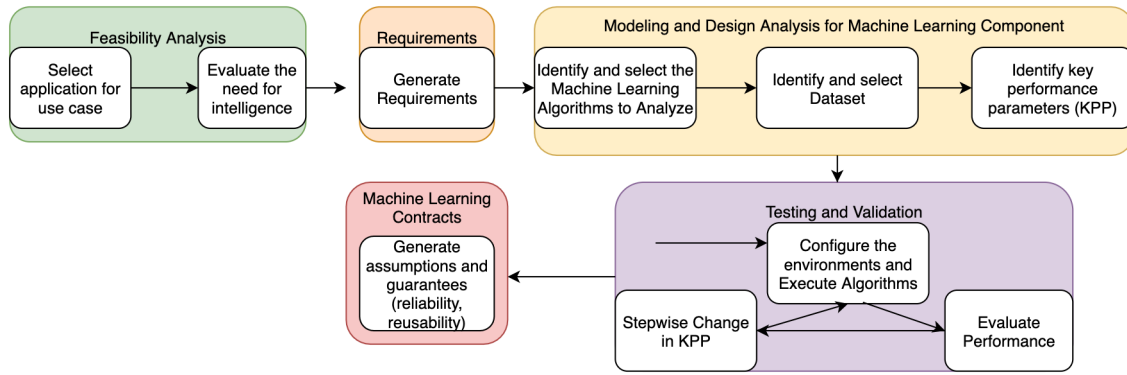


Fig. 1. Proposed SEGML Framework

selecting a dataset, one of the databases that help this field is the Modified National Institute of Standards and Technology database MNIST database; which is a combination of the training dataset taken from American Census Bureau employees, and the testing dataset which is taken from American high school students, that is the MNIST, original datasets. This database is used for training various image processing systems. It contains a large number of the training set and test set. The database has normalized and centered the size of the digits in a fixed-size image[9].

Following the selection of the dataset, the hyper parameters, identified from the ML algorithms. Hyper parameters are parameters whose value is used to control the learning process of a ML algorithm. From the hyper parameter, KPP were finalized based on the criteria which parameters can be changed without changing the architecture of the ML algorithm. In our experimentation, the execution environment selected was PyTorch, which is a machine learning framework using Python language. PyTorch uses dynamic computation and that allows the machine learning models to be expressed in idiomatic Python. PyTorch uses Tensor which is a multidimensional array and it accelerates mathematical operations. [10]

#### IV. ALGORITHMS INVESTIGATED

1) *Logistic Regression*: The logistic Regression is a statistical method for analyzing a dataset in which there are one or more independent variables that determine an outcome. In Logistic Regression, there are four classes of information in the output of Logistic Regression. First, defining the straight line showing the values of the relation between the dependent variable for every independent variable. Next, measuring how the dependent variable and independent variable are related to each other in the data. Then, showing how the data is drawn the relation. Finally, containing the estimation of values that is used for calculating some statistical significance such as regression line, the coefficients and termed standard errors [11].

In Logistic Regression, the outcome is measured with a dichotomous variable 1 or 0 which are the only outcomes [12]. This algorithm tries to find the decision boundary by

iterating over the training examples, trying to fit parameters that describe the decision boundary hyper surface equation [13].

2) *Neural Network*: The Neural Network is an algorithm that acts like a human neural network, that has the ability to collect and classify information according to a specific architecture. The network bears a strong resemblance to statistical methods such as curve fitting and regression analysis. The Neural Network contains several layers of interconnected nodes, each node is a perceptron which represents and simulate the ability of the brain to recognize and classify and resembles multiple linear regressions. The perceptron feeds the signal produced by a multiple linear regression into an activation function that may be nonlinear. The input layer collects input patterns. The output layer has classifications or output signals to which input patterns may map. To reduce the neural network's margin of error, hidden layers are used to fine-tune the input weightings [14].

#### A. Keys Performance Parameters

After investigating the algorithms and their implementation we identified the KPP, as performance parameters that can be easily investigated. KPPs were obtained from the hyper parameters represented in the code. These parameters were the learning rate, the epoch and the batch size. The learning rate is a tuning parameter that indicates how the objective function within a machine learning algorithm changes after each iteration of training to get closer to the objective. An epoch is one complete presentation of the dataset to be learned to a learning machine. The batch size in machine learning refers to the number of training examples that are used in one iteration. The other parameters such as, input size and number of classes could not be changed. As input size is the size of the image obtained from MNIST dataset it cannot be changed and number of classes represent the numbers from 0-9. It is critical to initially identify what parameters can be changed to evaluate the performance and what cannot be changed.

#### V. MACHINE LEARNING ALGORITHM EVALUATION

We evaluated the performance of the Logistic Regression and Neural Network algorithms by following a structured

systems/software engineering guided approach. We trained each algorithm on a dataset by varying the KPPs. The batch size varied from 50 to 150 with increments of 50. The variation in epoch was done in increments of 5 by first starting to train the algorithm on an epoch of 5 and continuing the change until the value of epoch reached 20. Each time the MNIST database was used as the dataset. We also increased the learning rate incrementally in multiples of 10, as indicated in the sequence 0.001, 0.002, 0.003, 0.01, 0.02, 0.03, 0.1, 0.2, 0.3.

During our experiments, we ran the Logistic Regression and Neural Network algorithms with each dataset. The performance of each algorithm showed different results. We will explain each algorithms performance down below.

#### A. Logistic Regression:

1) *First run on the dataset::* In training Logistic Regression algorithm during the first run on the dataset, the training time required for Logistic Regression was observed by changing the epoch and the learning rate. The first run is based on the batch size value of 50 and the learning rate varied between 0.001 to 0.3. On epoch 5, Logistic Regression took  $5.3 \times 10^{-5}$  (s) in training time with decreasing training time until it reached  $4.6 \times 10^{-5}$  (s) with learning rate 0.2 as it appears in Fig 2. On epoch 10, Logistic Regression did not show a consistent pattern but it took more time compared to epoch 5. On epoch 15, Logistic Regression did not show a consistent pattern but the time decreased and reached  $4.1 \times 10^{-5}$  (s). On epoch 20, the time that Logistic Regression took for training was around  $4.2 \times 10^{-5}$  (s). It decreased to  $4.0 \times 10^{-5}$  (s) with 0.001 learning rate and increased to  $4.4 \times 10^{-5}$  (s) with 0.02 learning rate as it shown in Fig 2.

On the other hand, the accuracy of Logistic Regression algorithms incrementally increased, as we increased the epoch and the learning rate. As in Fig 3, the accuracy started at 85% and increased until it reached 92%. Logistic Regression showed a consistent pattern in accuracy.

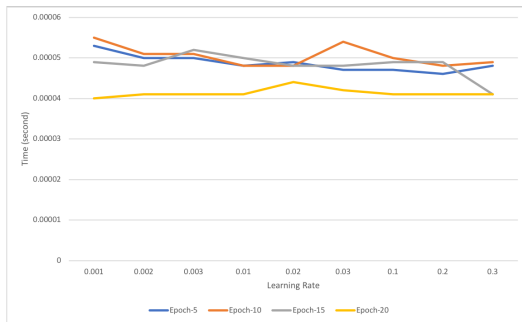


Fig. 2. Time performance of Logistic Regression With Batch Size 50

2) *Second run on the dataset::* In training Logistic Regression algorithm during the second run on the dataset, the training time required for Logistic Regression was observed by changing the epoch and the learning rate. The second run is based on the batch size value of 100 and the learning rate varied between 0.001 to 0.3. On epoch 5, Logistic Regression

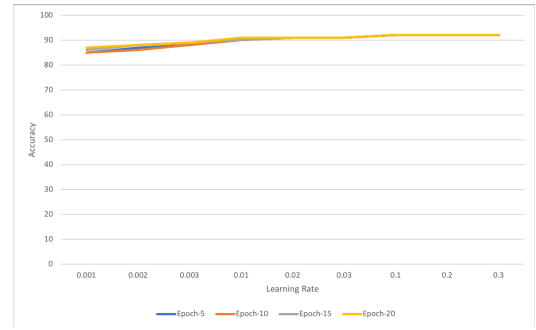


Fig. 3. Accuracy performance of Logistic Regression With Batch Size 50

did not show a consistent pattern in the training time as there was not a consistent increase or decrease. As it appears, in Fig 4, the time that Logistic Regression took for training with learning rate 0.001 was  $4.0 \times 10^{-5}$  (s) and with learning rate 0.01, it took less time. On epoch 10, Logistic Regression did not show a consistent pattern but it took less time compared to epoch 5. On epoch 15, Logistic Regression did not show a consistent pattern and showed close results to epoch 5. However, the only time that Logistic Regression took more time when we increased the learning rate to 0.3 and the time was  $5.9 \times 10^{-5}$  (s). On epoch 20, the time that Logistic Regression took for training with learning rate 0.001 was  $4.1 \times 10^{-5}$  (s) and with learning rate 0.03, it took less time. But, Logistic Regression took more time when we trained it with learning rate 0.3 as it appears in Fig 4.

On the other hand, the accuracy of Logistic Regression algorithms incrementally increased, as we increased the epoch and the learning rate. As in Fig 5, the accuracy started at 83% and increased until it reached 92%. Logistic Regression showed a consistent pattern in accuracy.

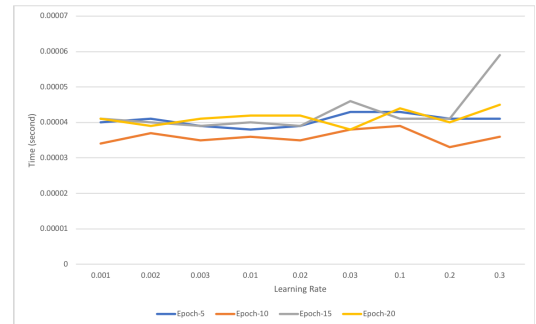


Fig. 4. Time performance of Logistic Regression With Batch Size 100

3) *Third run on dataset::* In training Logistic Regression algorithm on the third run on the dataset, the batch size was set to 150. In epoch 5, Logistic Regression training time was approximately the same as it appears in Fig 6. On epoch 10, Logistic Regression did not show a consistent pattern in the training time as there was not a consistent increase or decrease. On epoch 15, the training time was almost the same except with the learning rate 0.01, it took  $4.6 \times 10^{-5}$  (s). On epoch 20, the training time was around  $4.0 \times 10^{-5}$  (s). Logistic

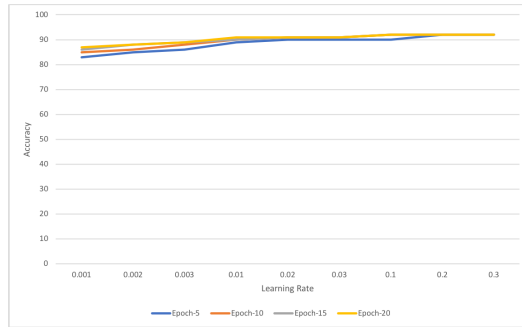


Fig. 5. Accuracy performance of Logistic Regression With Batch Size 100

Regression took  $4.4 \times 10^{-5}$  (s) with learning rate 0.3 and that was the long time that Logistic Regression took during epoch 20.

On the other hand, the accuracy of Logistic Regression algorithms incrementally increased, as we increased the epoch and the learning rate. As in Fig 7, the accuracy started at 80% and increased until it reached 92%. Logistic Regression showed a consistent pattern in accuracy.



Fig. 6. Time performance of Logistic Regression With Batch Size 150

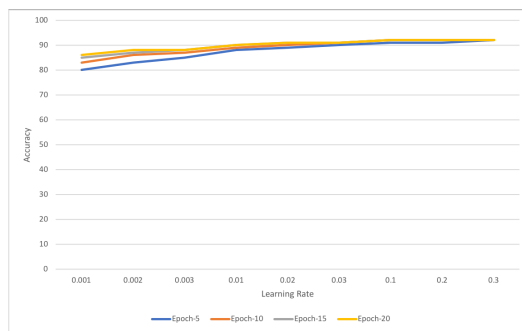


Fig. 7. Accuracy performance of Logistic Regression Batch Size 150

## B. Neural Network:

1) *First run on the dataset::* In training Neural Network algorithm during the first run on the dataset, the training time required for Neural Network was observed by changing the epoch and the learning rate. The first run is based on the batch size value of 100 and the learning rate varied between 0.001

to 0.3. On epoch 5, the time that Neural Network took for training was around  $4.4 \times 10^{-5}$  (s). It increased to  $4.8 \times 10^{-5}$  (s) with learning rate 0.003 and decreased to  $4.0 \times 10^{-5}$  (s) with learning rate 0.002. On epoch 10, the training time was between  $4.3 \times 10^{-5}$  (s) with learning rate 0.002 and  $4.9 \times 10^{-5}$  (s) with learning rate 0.03 as it appears in Fig 8. On epoch 15, the training time was varied and there was no pattern and it reached  $5.1 \times 10^{-5}$  (s) with learning rate 0.1. On epoch 20, the training time did not show a consistent pattern but it reached  $5.8 \times 10^{-5}$  (s) with learning rate 0.1 which was unexpected.

On the other hand, the accuracy of Neural Network algorithm incrementally decreased while we increased the learning rate. It was unexpected that the accuracy suddenly decreased when we reach the learning rate 0.2. As in Fig 9, the accuracy started at an excellent score of 98% and then decreased until it reached 11%. On epoch 10, the accuracy reached 9% with learning rate 0.3 and is the lowest accuracy during the experiment.

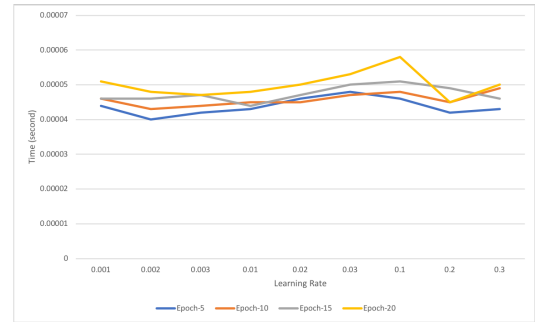


Fig. 8. Time performance of Neural Network With Batch Size 50

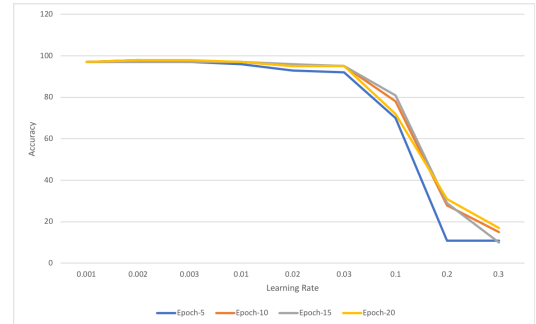


Fig. 9. Accuracy performance of Neural Network With Batch Size 50

2) *Second run on the dataset::* In training Neural Network algorithm during the second run on the dataset, the training time required for Neural Network was observed by changing the epoch and the learning rate. The second run is based on the batch size value of 100 and the learning rate varied between 0.001 to 0.3. On epoch 5, the time that Neural Network took for training was between  $3.4 \times 10^{-5}$  (s) with learning rate 0.003 and  $3.9 \times 10^{-5}$  (s) with learning rate 0.03. On epoch 10, Neural Network did not show a pattern in the training time as it appears in Fig 10. The training time varied between  $5.3 \times 10^{-5}$  (s) with learning rate 0.1 and  $3.7 \times 10^{-5}$  (s) with

learning rate 0.3. On epoch 15, the training time was varied and there was no pattern. On epoch 20, the training time also varied and was between  $5.0 \times 10^{-5}$  (s) with learning rate 0.02 and  $3.8 \times 10^{-5}$  (s) with learning rate 0.03.

On the other hand, the accuracy of Neural Network algorithm incrementally decreased while we increased the learning rate. It was unexpected that the accuracy suddenly decreased when we reach the learning rate 0.2. As in Fig 11, the accuracy started at an excellent score of 98% and then decreased until it reached 13%.

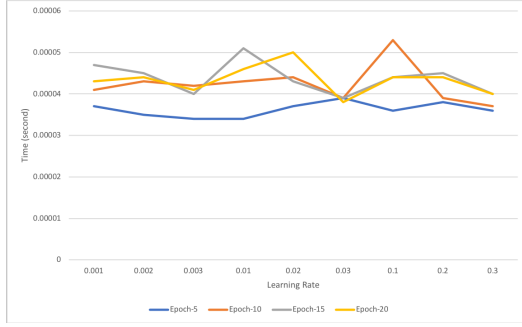


Fig. 10. Time performance of Neural Network With Batch Size 100



Fig. 11. Accuracy performance of Neural Network With Batch Size 100

3) *Third run on the dataset::* In training Neural Network algorithm during the third run on the dataset, the training time required for Neural Network was observed by changing the epoch and the learning rate. The third run is based on the batch size value of 150 and the learning rate varied between 0.001 to 0.3. On epoch 5, Neural Network took  $3.9 \times 10^{-5}$  (s) in training time and then was around  $4.0 \times 10^{-5}$  (s) as it appears in Fig 12. On epoch 10, Neural Network did not show a consistent pattern and it was similar to epoch 5 training time. On epoch 15, Neural Network did not show a consistent pattern but the time increased and reached  $4.2 \times 10^{-5}$  (s) with learning rate 0.3. On epoch 20, the time that Neural Network took for training was varied. As it appears in Fig 12, the training time for Neural Network started with  $5.3 \times 10^{-5}$  (s) and then it decreased to  $4.0 \times 10^{-5}$  (s) with 0.001 learning rate. Then it jumped back to  $5.3 \times 10^{-5}$  (s) with learning rate 0.2.

On the other hand, the accuracy of Neural Network algorithm incrementally decreased while we increased the learning

rate. It was unexpected that the accuracy suddenly decreased when we reach the learning rate 0.2. As in Fig 13, the accuracy started at an excellent score at 97% and then decreased until it reached 15%.

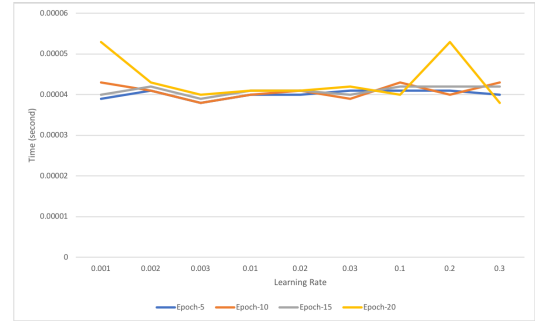


Fig. 12. Time performance of Neural Network with Batch Size 150

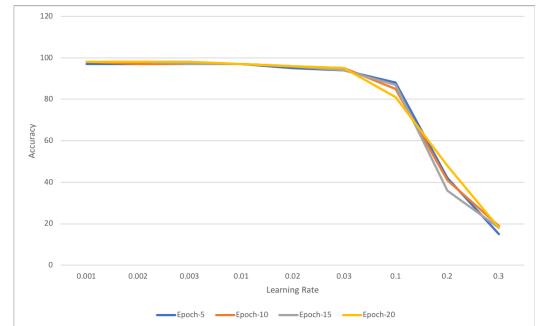


Fig. 13. Accuracy performance of Neural Network with Batch Size 150

### C. Comparing the evaluation:

We evaluate the performance of both Logistic Regression and Neural Network algorithms, by comparing the results between these two algorithms in the time and the accuracy.

1) *Comparing first run on dataset::* In training both algorithms on the first run on the dataset with batch size 50, we can see from Fig 2 and Fig 8 that Logistic Regression took less time in training than Neural Network especially with epoch 20 as it shown in Fig 2 and Fig 8. The results comparing accuracy was surprising because Neural Network showed excellent performance in the beginning but then it started to decline until it reached 9% with epoch 10. On the other hand, for Logistic Regression the performance increased until it reached 92% as shown in Fig 3 and Fig 9.

2) *Comparing second run on dataset::* In training both algorithms on the second run on the dataset with batch size 100, we can see from Fig 4 and Fig 10 that in some cases Neural Network took less time in training than Logistic Regression but overall Logistic Regression took less time. Comparison of accuracy showed surprising results because Neural Network did an excellent job in the beginning but then it started to decline until it reached 10% with epoch 15. On the other hand, Logistic Regression did well and started to increase until it reached 92% as shown in Fig 5 and Fig 11



3) *Comparing third run on dataset*:: In training both algorithms on the third run on the dataset with batch 150, we can see from Fig 6 and Fig 12 that time fluctuated and in some cases Logistic Regression took less time in training than Neural Network and in some cases Neural Network took less time. The results comparing accuracy showed that Neural Network performed better in the beginning but then it started to decline until it reached 15% with epoch 5. On the other hand, Logistic Regression exhibited continuous increase until it reached 92% as shown in Fig 7 and Fig 13

After the performance evaluation is complete, the machine learning contract associated with the selected intelligent algorithm will include the assumptions and guarantees based on the chosen algorithm. For example, a contract for the intelligent algorithm will be assumptions on the learning rate, epoch, batch size, with guarantees on the accuracy, time, which can then be included for the overall analysis of the system/software for reliability or reusability. Once an intelligent algorithm is trained and the contracts are identified the trained intelligent algorithm can be reused or integrated for similar purpose within systems used for different applications.

## VI. RESULTS

With batch size 50, Logistic Regression algorithm exhibited improvement in accuracy from 85% to 88%, as the value of Epoch varied from 5 to 20, at the learning rate of 0.001. As the learning rate was increased the accuracy achieved improved to a value of 92%, which was the same irrespective of the value of the Epoch. So, the change in Epoch did not have any effect on the final accuracy achieved. Neural Network exhibited an accuracy of 97% for all values of Epoch until the learning rate became larger than 0.03. So, the performance boundary identified for Neural Network is the learning rate of 0.03. The variation in training time did not exhibit any pattern for any of the algorithms tested.

After the evaluation of both Logistic Regression and Neural Network algorithms, it is evident that Logistic Regression shows a more stable performance in recognizing handwritten digits when the KPP observed are Batch Size, Epoch and Learning rate. This is because, the change in performance of Logistic Regression is slow and gradual as expected, whereas that of Neural network is drastic after a certain value of the learning parameter. The value of Learning rate plays an important role in training, an optimal value of the learning rate gives accurate results, whereas a very high value causes unstable learning, resulting in inaccuracy. This has been specially studied and documented for Neural Networks by Wilson [15] and Arun K [16].

The constraints on the requirements in integrating Logistic Regression will not be as stringent as that required for Neural Networks. For example, if Neural Networks is integrated the requirements for the KPP (Learning rate, Epoch) need to be strictly followed such that the value of learning parameter should not be above 0.03. By following the SEGML process we can come up with the performance boundary parameters related to number of times we need to train the algorithm on a

training set and what should the learning rate be as we try to decide upon the training time or accuracy we want to achieve with the intelligent algorithm. Logistic Regression takes less time in training and recognizing handwritten digits in most of the cases that we examined. In addition, Logistic Regression perform better as we trained it and it gets accurate overtime. As a result Logistic Regression seems to have more stable performance compared to Neural Network.

## VII. CONCLUSION

In this paper, we demonstrated a structured systems/software engineering approach to develop contracts for intelligent algorithms by evaluating the performance of intelligent algorithms. We identified that one of the algorithms is more stable and shows expected improvement in performance, whereas the other, starts with good performance but then deteriorates. Thus, this process helps in engineering of ML algorithms. Additionally, these structured approaches indicate the kind of interaction that should happen between a system/software engineer and a data scientist as intelligent algorithms are integrated in reliable intelligent products. Future work, will be to extend the SEGML approach to evaluate more algorithms

## REFERENCES

- [1] H. Ahamed, I. Alam, and M. Islam, "Handwritten digit recognition system based on lrm and svm algorithm," 01 2019.
- [2] M. Agarwal, Shalika, V. Tomar, and P. Gupta, "Handwritten character recognition using neural network and tensor flow," vol. 8. Blue Eyes Intelligence Engineering Sciences Publication, 2019, p. 1445–1448.
- [3] J. Chen, "Neural network," 2019. [Online]. Available: <https://www.investopedia.com/terms/n/neuralnetwork.asp>
- [4] G. Hulten, *Building Intelligent Systems: A Guide to Machine Learning Engineering*. Apress, 2018.
- [5] R. Caruana and A. Niculescu-Mizil, "An empirical comparison of supervised learning algorithms," in *Proceedings of the 23rd International Conference on Machine Learning*, ser. ICML '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 161–168. [Online]. Available: <https://doi.org/10.1145/1143844.1143865>
- [6] W. Han, L. Jiang, T. Lu, and X. Zhang, "Comparison of machine learning algorithms for software project time prediction," *International Journal of Multimedia and Ubiquitous Engineering*, vol. 10, pp. 1–8, 10 2015.
- [7] D. Lewis, C. Info, L. Studies, and M. Ringuette, "A comparison of two learning algorithms for text categorization," *Third Annual Symposium on Document Analysis and Information Retrieval*, 10 1996.
- [8] D. Y. Perwey and A. Chaturvedi, "Neural networks for handwritten english alphabet recognition," *International Journal of Computer Applications (0975 – 8887)*, vol. Volume 20– No.7., pp. 1–5, 04 2011.
- [9] M. Bhubé, "Mnist — digits classification with keras," 2016. [Online]. Available: <https://medium.com/@mjhbobe/mnist-digits-classification-with-keras-ed6c2374bd0e>
- [10] E. Stevens, L. Antiga, and T. Viehmann, *Deep Learning with PyTorch*. Manning Publications Co, 2019.
- [11] J. M. Hilbe, "Practical guide to logistic regression," 2016.
- [12] M. S. Ltd., "Logistic regression," 2020. [Online]. Available: [https://www.medcalc.org/manual/logistic\\_regression.php](https://www.medcalc.org/manual/logistic_regression.php)
- [13] I. Uhliarik, "Handwritten character recognition using machine learning methods," 2013.
- [14] C. Aggarwal, "Neural networks and deep learning: A textbook," 2018.
- [15] D. R. Wilson and T. R. Martinez, "The need for small learning rates on large problems," in *IJCNN'01. International Joint Conference on Neural Networks. Proceedings (Cat. No.01CH37222)*, vol. 1, 2001, pp. 115–119 vol.1.
- [16] A. K., "Understanding learning rate in machine learning," 2020. [Online]. Available: <https://www.mygreatlearning.com/blog/understanding-learning-rate-in-machine-learning/>